

§ 6.2 抽象数据类型

6.2.1 操作接口

作为抽象数据类型，图支持的操作接口分为边和顶点两类，分列于表6.1和表6.2。

表6.1 图ADT支持的边操作接口

| 操 作 接 口      | 功 能 描 述        |
|--------------|----------------|
| e()          | 边总数 E          |
| exist(v, u)  | 判断联边(v, u)是否存在 |
| insert(v, u) | 引入从顶点v到u的联边    |
| remove(v, u) | 删除从顶点v到u的联边    |
| status(v, u) | 边(v, u)的状态     |
| edge(v, u)   | 边(v, u)对应的数据域  |
| weight(v, u) | 边(v, u)的权重     |

表6.2 图ADT支持的顶点操作接口

| 操 作 接 口                     | 功 能 描 述       |
|-----------------------------|---------------|
| n()                         | 顶点总数 V        |
| insert(v)                   | 在顶点集V中插入新顶点v  |
| remove(v)                   | 将顶点v从顶点集中删除   |
| inDegree(v)<br>outDegree(v) | 顶点v的入度、出度     |
| firstNbr(v)                 | 顶点v的首个邻接顶点    |
| nextNbr(v, u)               | 在v的邻接顶点中，u的后继 |
| status(v)                   | 顶点v的状态        |
| dTime(v)、fTime(v)           | 顶点v的时间标签      |
| parent(v)                   | 顶点v在遍历树中的父节点  |
| priority(v)                 | 顶点v在遍历树中的权重   |

6.2.2 Graph模板类

代码6.1以抽象模板类的形式，给出了图ADT的具体定义。

```
1 typedef enum { UNDISCOVERED, DISCOVERED, VISITED } VStatus; //顶点状态
2 typedef enum { UNDETERMINED, TREE, CROSS, FORWARD, BACKWARD } EStatus; //边状态
3
4 template <typename Tv, typename Te> //顶点类型、边类型
5 class Graph { //图Graph模板类
6 private:
7     void reset() { //所有顶点、边的辅助信息复位
8         for (int i = 0; i < n; i++) { //所有顶点的
9             status(i) = UNDISCOVERED; dTime(i) = fTime(i) = -1; //状态, 时间标签
10            parent(i) = -1; priority(i) = INT_MAX; // ( 在遍历树中的 ) 父节点, 优先级数
11            for (int j = 0; j < n; j++) //所有边的
12                if (exists(i, j)) status(i, j) = UNDETERMINED; //状态
13        }
14    }
15    void BFS(int, int&); // ( 连通域 ) 广度优先搜索算法
16    void DFS(int, int&); // ( 连通域 ) 深度优先搜索算法
17    void BCC(int, int&, Stack<int>&); // ( 连通域 ) 基于DFS的双连通分量分解算法
18    bool TSort(int, int&, Stack<Tv>*&); // ( 连通域 ) 基于DFS的拓扑排序算法
19    template <typename PU> void PFS(int, PU); // ( 连通域 ) 优先级搜索框架
```

```

20 public:
21 // 顶点
22     int n; //顶点总数
23     virtual int insert(Tv const&) = 0; //插入顶点, 返回编号
24     virtual Tv remove(int) = 0; //删除顶点及其关联边, 返回该顶点信息
25     virtual Tv& vertex(int) = 0; //顶点v的数据 (该顶点的确存在)
26     virtual int inDegree(int) = 0; //顶点v的入度 (该顶点的确存在)
27     virtual int outDegree(int) = 0; //顶点v的出度 (该顶点的确存在)
28     virtual int firstNbr(int) = 0; //顶点v的首个邻接顶点
29     virtual int nextNbr(int, int) = 0; //顶点v的 (相对于顶点j的) 下一邻接顶点
30     virtual VStatus& status(int) = 0; //顶点v的状态
31     virtual int& dTime(int) = 0; //顶点v的时间标签dTime
32     virtual int& fTime(int) = 0; //顶点v的时间标签fTime
33     virtual int& parent(int) = 0; //顶点v在遍历树中的父亲
34     virtual int& priority(int) = 0; //顶点v在遍历树中的优先级数
35 // 边: 这里约定, 无向边均统一转化为方向互逆的一对有向边, 从而将无向图视作有向图的特例
36     int e; //边总数
37     virtual bool exists(int, int) = 0; //边(v, u)是否存在
38     virtual void insert(Te const&, int, int, int) = 0; //在顶点v和u之间插入权重为w的边e
39     virtual Te remove(int, int) = 0; //删除顶点v和u之间的边e, 返回该边信息
40     virtual EStatus& status(int, int) = 0; //边(v, u)的状态
41     virtual Te& edge(int, int) = 0; //边(v, u)的数据 (该边的确存在)
42     virtual int& weight(int, int) = 0; //边(v, u)的权重
43 // 算法
44     void bfs(int); //广度优先搜索算法
45     void dfs(int); //深度优先搜索算法
46     void bcc(int); //基于DFS的双连通分量分解算法
47     Stack<Tv>* tSort(int); //基于DFS的拓扑排序算法
48     void prim(int); //最小支撑树Prim算法
49     void dijkstra(int); //最短路径Dijkstra算法
50     template <typename PU> void pfs(int, PU); //优先级搜索框架
51 };

```

代码6.1 图ADT操作接口

仍为简化起见, 这里直接开放了变量n和e。除以上所列的操作接口, 这里还明确定义了顶点和边可能处于的若干状态, 并通过内部接口reset()复位顶点和边的状态。

图的部分基本算法在此也以操作接口的形式供外部用户直接使用, 比如广度优先搜索、深度优先搜索、双连通分量分解、最小支撑树、最短路径等。为求解更多的具体应用问题, 读者可照此模式, 独立地补充相应的算法。

就功能而言, 这些算法均超脱于图结构的具体实现方式, 借助统一的顶点和边ADT操作接口直接编写。尽管如此, 正如下即将看到的, 图算法的时间、空间性能, 却与图结构的具体实现方式紧密相关, 在这方面的理解深度, 也将反映和决定我们对图结构的驾驭与运用能力。

## § 6.3 邻接矩阵

### 6.3.1 原理

邻接矩阵 (adjacency matrix) 是图ADT最基本的实现方式, 使用方阵 $A[n][n]$ 表示由 $n$ 个顶点构成的图, 其中每个单元, 各自负责描述一对顶点之间可能存在的邻接关系, 故此得名。

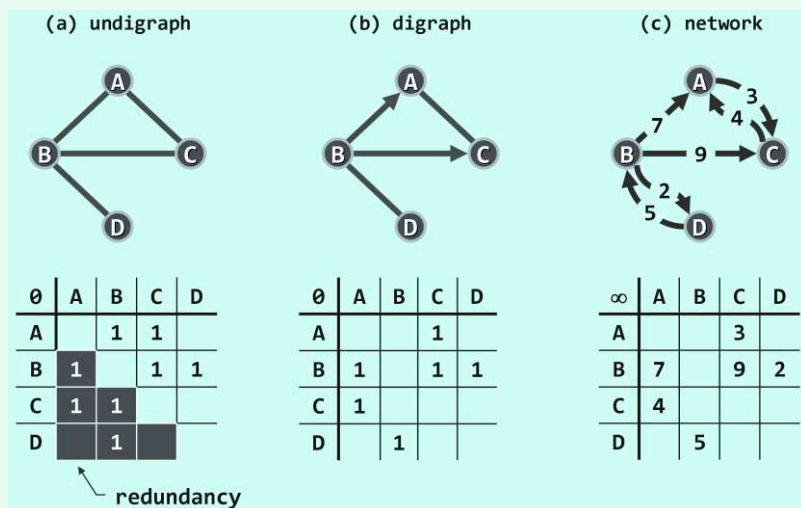


图6.5 邻接矩阵 (空白单元对应的边不存在, 其统一取值标注于矩阵最左上角)

对于无权图, 存在 (不存在) 从顶点 $u$ 到 $v$ 的边, 当且仅当 $A[u][v] = 1$  ( $0$ )。图6.5(a)和(b)即为无向图和有向图的邻接矩阵实例。

这一表示方式, 不难推广至带权网络。此时如图(c)所示, 矩阵各单元可从布尔型改为整型或浮点型, 记录所对应边的权重。对于不存在的边, 通常统一取值为 $\infty$ 或 $0$ 。

### 6.3.2 实现

基于以上原理与构思实现的图结构如代码6.2所示。

```
1 #include "../Vector/Vector.h" //引入向量
2 #include "../Graph/Graph.h" //引入图ADT
3
4 template <typename Tv> struct Vertex { //顶点对象 (为简化起见, 并未严格封装)
5     Tv data; int inDegree, outDegree; VStatus status; //数据、出入度数、状态
6     int dTime, fTime; //时间标签
7     int parent; int priority; //在遍历树中的父节点、优先级数
8     Vertex(Tv const& d = (Tv) 0) : //构造新顶点
9         data(d), inDegree(0), outDegree(0), status(UNDISCOVERED),
10         dTime(-1), fTime(-1), parent(-1), priority(INT_MAX) {} //暂不考虑权重溢出
11 };
12
13 template <typename Te> struct Edge { //边对象 (为简化起见, 并未严格封装)
14     Te data; int weight; EStatus status; //数据、权重、状态
15     Edge(Te const& d, int w) : data(d), weight(w), status(UNDETERMINED) {} //构造新边
16 };
17
```